

Kunal patil

PYTHON SETS

Unordered Collection of Unique Items

① What is a Set?

A set is a collection of **unique elements** in Python. It does not allow **duplicate values** and is **unordered**.

② Important Points

- Sets do **not** allow **duplicate** values
- Elements are **not ordered**
- Defined using curly braces **{ }**

③ Example

If duplicate values are added, Python automatically **removes** them.

$\{1, 2, \cancel{2}, 3\} \rightarrow \{1, 2, 3\}$

Set Removes Duplicates

```
⑤ my_set = {1, 2, 2, 3}
   print(my_set)
```

Output → **{1, 2, 3}**

- Set is an **unordered collection** in Python.
- It stores only **unique values**.
- Duplicate elements are automatically removed.
- Set is **mutable**, so elements can be added or removed.
- Indexing and slicing are not supported.
- Set uses **hashing internally** for fast searching.
- Searching in set is generally:

$O(1)$

- Only **hashable (immutable)** elements are allowed inside a set.
- Sets are mainly used for:
 - duplicate removal
 - fast lookup
 - mathematical operations

1. Using Curly Braces { }

```
In [25]: s = {1, 2, 3}
         print(s)
         type(s)
```

```
{1, 2, 3}
```

```
Out[25]: set
```

2. Using set() Function

```
In [29]: s = set([1, 2, 3])
         print(s)
         type(s)
```

```
{1, 2, 3}
```

```
Out[29]: set
```

Empty Set Syntax

Correct Way

```
In [48]: s = set()

         print(type(s))

<class 'set'>
```

`{}` creates an empty dictionary, not a set.

```
In [42]: s = {}
         s
         print(type(s))

<class 'dict'>
```

3.String to Set

```
In [58]: s = set("PYTHON")

         print(s)

{'P', 'O', 'T', 'Y', 'H', 'N'}
```

Tuple to Set

```
In [73]: t = set((10,20,30))

         print(t)
         print(type(t))

{10, 20, 30}
<class 'set'>
```

```
In [75]: t = set(10,20,30)

         print(t)
         print(type(t))
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[75], line 1
----> 1 t = set(10,20,30)
      3 print(t)
      4 print(type(t))

TypeError: set expected at most 1 argument, got 3
```

```
In [ ]:
```

4.Nested Set Concept

A normal set cannot contain another mutable set.

Why?

Because:

- set is mutable
- mutable objects are unhashable
- sets use hashing internally

Searching complexity:

$O(1)$

```
In [79]: s = {{1,2}, {3,4}}
print(s)
print(type(s))
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[79], line 1
----> 1 s = {{1,2}, {3,4}}
      2 print(s)
      3 print(type(s))

TypeError: unhashable type: 'set'
```

```
In [81]: t = {{1,2}, (3,4)}
print(t)
print(type(t))
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[81], line 1
----> 1 t = {{1,2}, (3,4)}
      2 print(t)
      3 print(type(t))

TypeError: unhashable type: 'set'
```

```
In [85]: t = {(1,2), (3,4)}
print(t)
print(type(t))
```

```
{(1, 2), (3, 4)}
<class 'set'>
```

```
In [ ]:
```

Valid Nested Set Using frozenset

```
In [91]: s = {frozenset([1,2]), frozenset([3,4])}

print(s)
```

```
{frozenset({3, 4}), frozenset({1, 2})}
```

5. Immutable Elements Inside Set

Only immutable (hashable) elements are allowed inside a set.

```
In [104... s = {1,2,3}
K = {"Python", "Java"}
P = {(1,2), (3,4)}
SET= {(1,3,4), ("Python", "Java", "NLP"), (1,2.4,2+3j)}

print(s,K,P,SET)
print(s)
print(P)
print(SET)
print(type(SET))
```

```
{1, 2, 3} {'Python', 'Java'} {(1, 2), (3, 4)} {(1, 2.4, (2+3j)), ('Python', 'Java', 'NLP'), (1, 3, 4)}
{1, 2, 3}
{(1, 2), (3, 4)}
{(1, 2.4, (2+3j)), ('Python', 'Java', 'NLP'), (1, 3, 4)}
<class 'set'>
```

```
In [ ]:
```

Not Allowed Elements

```
In [107... s = {[1,2]}
print(s)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[107], line 1
----> 1 s = {[1,2]}
      2 print(s)

TypeError: unhashable type: 'list'
```

```
In [109... s = {{1,2}}
print(s)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[109], line 1
----> 1 s = {{1,2}}
      2 print(s)

TypeError: unhashable type: 'set'
```

```
In [ ]:
```

6. Set from List

```
In [112... s = set([1,2,2,3])
print(s)
```

```
{1, 2, 3}
```

```
In [114... names = set(["Kunal", "Rahul", "Kunal"])
print(names)
```

```
{'Rahul', 'Kunal'}
```

Set from String

```
In [123... k = set("HELLO")
s = set("Vangipurapu Venkata Sai Laxman")
print(s)
print(k) # duplicate are remove
```

```
{'a', 'p', 'u', 'n', 'e', 'k', 'V', 'i', 'm', 'x', 'L', 'S', 'g', 't', ' ', 'r'}
{'E', 'O', 'H', 'L'}
```

Set from Dictionary

By default, set takes only dictionary keys.

```
In [126... d = {"a":1, "b":2}
s = set(d)
print(s)
```

```
{'a', 'b'}
```

```
In [132... d = {"a":1, "b":2}
s = set(d.values())
k= set(d.keys())
print(s)
print(k)
```

```
{1, 2}
{'a', 'b'}
```

Set Functions

Python contains the following methods to be used with the sets. Those are

- **len(set)**
- **max(set)**
- **min(set)**
- **sum(set)**
- **sorted(set)**
- **set()**
- **add()**
- **update()**
- **discard()**
- **remove()**
- **pop()**
- **clear()**

len

```
In [4]: A = {10, 20, 30, 40, 50}
        print(len(A))
```

5

```
In [6]: print(max(A))
```

50

```
In [8]: print(min(A))
```

10

```
In [10]: print(sum(A))
```

150

sorted()

```
In [22]: s = {5, 2, 9, 1, 7}
        result = sorted(s)
        print(result)
```

[1, 2, 5, 7, 9]

```
In [24]: print(sorted(s, reverse=True))
```

[9, 7, 5, 2, 1]

```
In [26]: s = {5, 2, 1}
```

```
s.sort()
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[26], line 3
      1 s = {5, 2, 1}
----> 3 s.sort()
```

AttributeError: 'set' object has no attribute 'sort'

No, sort() method is not used in set. Because a set is unordered, it does not support the sort() method.

add() in Python Set

`add()` method is used to add one single element into a set.

- ✓ `add()` adds only one element
- ✓ Modifies original set
- ✓ Duplicate values ignored
- ✓ Returns `None`
- ✓ Average Time Complexity: $O(1)$

```
In [30]: s = {1, 2, 3}
s.add(4)
print(s)
```

```
{1, 2, 3, 4}
```

```
In [34]: s = {1, 2, 3}
s.add(2)
print(s)#2 already exists, so no duplicate added
```

```
{1, 2, 3}
```

```
In [36]: s = {"Python", "Java"}
s.add("AI")
print(s)
```

```
{'Python', 'Java', 'AI'}
```

```
In [38]: s = {1, 2}
s.add((3, 4))
print(s)
```

```
{1, 2, (3, 4)}
```

```
In [40]: s = {1, 2}
s.add([3, 4])
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[40], line 3
      1 s = {1, 2}
----> 3 s.add([3, 4])

TypeError: unhashable type: 'list'
```

update

`update()` method is used to add multiple elements into a set.

- ✓ `update()` adds multiple elements
- ✓ Accepts iterable objects
- ✓ Original set is modified
- ✓ Duplicate values are ignored
- ✓ Average Time Complexity: $O(n)$

```
In [46]: s = {1, 2, 3}
s.update([4, 5, 6])
print(s)
```

```
{1, 2, 3, 4, 5, 6}
```

```
In [48]: s = {1, 2}
s.update((3, 4))
print(s)
```

```
{1, 2, 3, 4}
```

```
In [50]: A = {1, 2}
        B = {3, 4}

        A.update(B)

        print(A)
```

```
{1, 2, 3, 4}
```

```
In [52]: s = {1}

        s.update([2,3], (4,5), {6,7})

        print(s)
```

```
{1, 2, 3, 4, 5, 6, 7}
```

```
In [67]: s = {1}
        print(s.add(2))

        s.update(2)

        print(s)
```

```
None
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[67], line 4
      2 print(s.add(2))
      3 s.add(2)
----> 4 s.update(2)
      6 print(s)

TypeError: 'int' object is not iterable
```

```
In [71]: s.add(2)
        s
```

```
Out[71]: {1, 2}
```

Difference: add() vs update()

Feature	add()	update()
Adds	One element	Multiple elements
Argument	Single value	Iterable
Example	<code>s.add(5)</code>	<code>s.update([5,6])</code>

remove() in Python Set

remove() method is used to delete a specific element from a set.

- ✓ Removes specified element
- ✓ Modifies original set
- ✓ Returns `None`
- ✓ Raises `KeyError` if element absent
- ✓ Average Time Complexity: `O(1)`

```
In [75]: s = {1, 2, 3}

        s.remove(2)

        print(s)
```

```
{1, 3}
```

```
In [77]: s = {1, 2, 3}

        s.remove(5)
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[77], line 3
      1 s = {1, 2, 3}
----> 3 s.remove(5)

KeyError: 5

```

discard() in Python Set

discard() method is used to remove an element from a set. If the element is not present, it does not give an error.

- ✓ Removes specified element
- ✓ No error if element absent
- ✓ Modifies original set
- ✓ Returns `None`
- ✓ Average Time Complexity: $O(1)$

```

In [81]: s = {1, 2, 3}

s.discard(2)

print(s)

```

```
{1, 3}
```

```

In [83]: s = {1, 2, 3}

s.discard(5)

print(s)

```

```
{1, 2, 3}
```

Feature	<code>remove()</code>	<code>discard()</code>
Element Missing	✗ KeyError	✓ No Error
Removes Element	✓ Yes	✓ Yes
Return Value	None	None

pop() in Python Set

pop() method removes and returns a random element from a set. Because sets are unordered, you cannot predict which element will be removed.

- ✓ Removes random element
- ✓ Returns removed value
- ✓ No argument allowed
- ✓ Raises `KeyError` on empty set
- ✓ Average Time Complexity: $O(1)$

```

In [90]: s = {1, 2, 3}

x = s.pop()

print(x)
print(s)

```

```

1
{2, 3}

```

```

In [92]: s = set()

s.pop()

```



```
-----
KeyError                                Traceback (most recent call last)
Cell In[92], line 3
      1 s = set()
----> 3 s.pop()

KeyError: 'pop from an empty set'
```

```
In [94]: s = {1, 2, 3}

        s.pop(1)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[94], line 3
      1 s = {1, 2, 3}
----> 3 s.pop(1)

TypeError: set.pop() takes no arguments (1 given)
```

```
In [96]: s = {1, 2, 3}

        s.pop(1)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[96], line 3
      1 s = {1, 2, 3}
----> 3 s.pop(1)

TypeError: set.pop() takes no arguments (1 given)
```

Difference: List `pop()` vs Set `pop()`

Feature	List <code>pop()</code>	Set <code>pop()</code>
Removes	By index	Random element
Ordered	Yes	No
Argument Allowed	Yes	No

clear

`clear()` method removes all elements from a set and makes it empty.

```
In [103... s = {1, 2, 3, 4}

          s.clear()

          print(s)
          print(len(s))
```

```
set()
0
```

```
In [105... s = {1, 2}

          del s

          print(s)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[105], line 5
      1 s = {1, 2}
      3 del s
----> 5 print(s)

NameError: name 's' is not defined
```

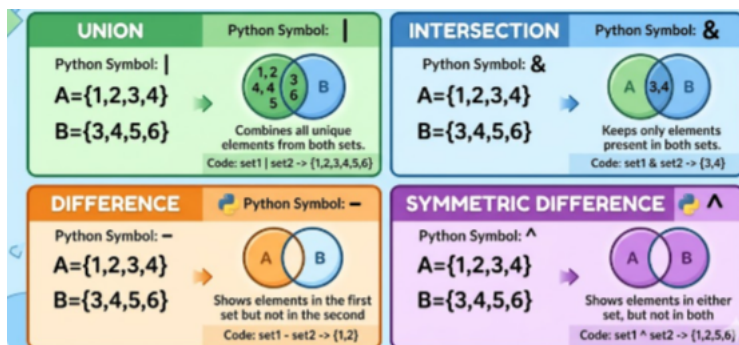
```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

In []:

Python Set Operations



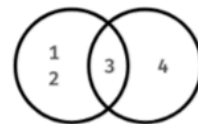
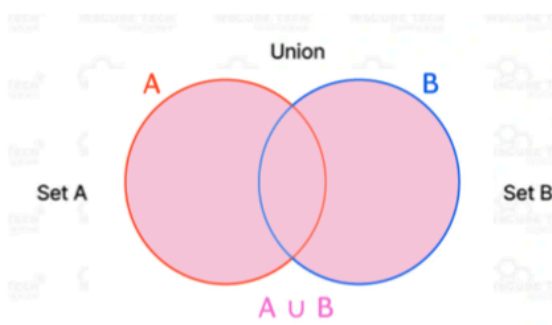
Python Set Operations are mathematical operations performed between two or more sets.

These operations are used to:

- combine sets
- find common elements
- find unique elements
- compare data

Union Operation

The Union Operator in Python Set is used to combine all unique elements from two or more sets.



$$A = \{1, 2, 3\}, B = \{3, 4\}$$

$$A \cup B$$

Meaning:

- all elements from A
- all elements from B
- duplicates removed

Why Duplicates Are Removed?

Because sets store only unique hashable elements

In [9]: `##syntax`

1. Using Union Operator |

Python

```
set1 | set2
```

2. Using union() Method

Syntax

Python

```
set1.union(set2)
```

```
In [30]: a = {1,2,3}
        b = {3,4,5}

        print(a | b)
```

```
{1, 2, 3, 4, 5}
```

```
In [16]: # create two sets
        a = {1,2,4}
        b = {3,5,7,1}
        # union between two sets
        a.union(b)
```

```
Out[16]: {1, 2, 3, 4, 5, 7}
```

```
In [20]: A = {1,2,3,4,5}
        B = {4,5,6,7,8}
        C = {8,9,10}
```

```
In [22]: A | B # Union of A and B (ALL elements from both sets. NO DUPLICATES)
```

```
Out[22]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [24]: print(a.union(b))

        {1, 2, 3, 4, 5, 7}
```

```
In [26]: set_one = {2, 3, 4, 5, 6}
        set_two = {0, 3, 6, 9, 12}

        print("Union of set-one and set-two is:", set_one.union(set_two))
```

```
Union of set-one and set-two is: {0, 2, 3, 4, 5, 6, 9, 12}
```

```
In [28]: # Initializing sets
        first_set = {1, 3, 5, 7, 9}
        second_set = {2, 4, 6, 8, 10}
        # Getting a union of a set and a List
        first_set.union([2, 4, 6, 8, 10])
```

```
Out[28]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
In [32]: #Union with Multiple Sets
        A = {1,2}
        B = {3,4}
        C = {5,6}
```

```
print(A.union(B,C))
```

```
{1, 2, 3, 4, 5, 6}
```

```
In [34]: #Union with List Converted to Set
        A = set([1,2,3])
        B = set([3,4,5])

        print(A.union(B))
        A|B
```

```
{1, 2, 3, 4, 5}
```

Out[34]: {1, 2, 3, 4, 5}

```
In [36]: A = set()
B = {1,2,3}

print(A.union(B))

{1, 2, 3}
```

```
In [38]: A = {1}
B = {2}
C = {3}

print(A | B | C)

{1, 2, 3}
```

```
In [44]: A = {1}
B = {2}
C = {1,2,3,(4,5)}
```

```
In [46]: print(A | B | C)

{1, 2, 3, (4, 5)}
```

update()

```
In [49]: A = {1,2}
B = {3,4}

A.update(B)

print(A)
print(B)

{1, 2, 3, 4}
{3, 4}
```

```
In [54]: A = {1,2}
B = {3,4}
c={2,4,"kunal"}
A.update(B,c)

print(A)
print(B)
print (c)

{1, 2, 3, 4, 'kunal'}
{3, 4}
{2, 4, 'kunal'}
```

Difference Between union() and update()

union()

Returns new set

Original unchanged

update()

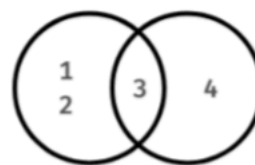
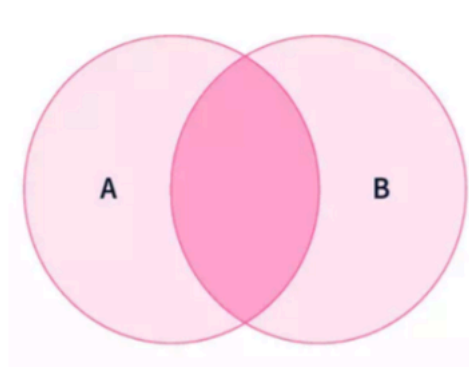
Modifies original set

Original changes



Intersection

The Intersection operation returns the common elements present in two or more sets.



$$A = \{1, 2, 3\}, B = \{3, 4\}$$

$$A \cap B = \{3\}$$

Set Intersection

Syntax

Using Operator

Python

`A & B`

Using Method

Python

`A.intersection(B)`

```
In [64]: A = {1,2,3,4}
         B = {3,4,5,6}

         print(A & B)
         print(A.intersection(B))
```

```
{3, 4}
{3, 4}
```

```
In [ ]:
```

```
In [67]: A = {"Python", "Java", "C++"}
         B = {"Java", "C"}

         print(A.intersection(B))
```

```
{'Java'}
```

```
In [69]: A = {1,2,3,4}
         B = {2,3,5}
         C = {2,3,6}

         print(A.intersection(B,C))
```

```
{2, 3}
```

```
In [71]: set_1 = {"a", "b", "c"}
         set_2 = {"c", "d", "a"}
         set_3 = {"d", "g", "c"}

         # Finding intersection using intersection() method:
         print("Intersection of three sets is:", set_1.intersection(set_2, set_3))
```

```
Intersection of three sets is: {'c'}
```

```
In [73]: set_1 = {"a", "b", "c"}
         set_2 = {"c", "d", "a"}
```

```
set_3 = {"d", "g", "c"}

# Finding intersection using intersection operator:
print("Intersection of three sets is:", set_1 & set_1 & set_3)
```

Intersection of three sets is: {'c'}

intersection_update()

intersection_update() modifies the original set by keeping only common elements.

```
set1.intersection_update(set2)
```

```
In [77]: A = {1,2,3,4}
        B = {3,4,5,6}

        A.intersection_update(B)

        print(A)
```

{3, 4}

```
In [79]: A = {1,2,3,4,5}
        B = {2,3,4}
        C = {3,4,6}

        A.intersection_update(B, C)

        print(A)
```

{3, 4}

intersection()

Returns new set

Original unchanged

Return type = set

intersection_update()

Modifies original set

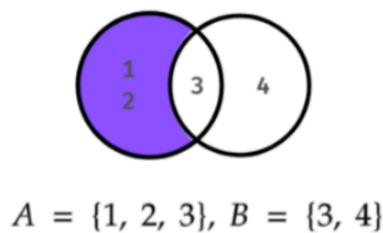
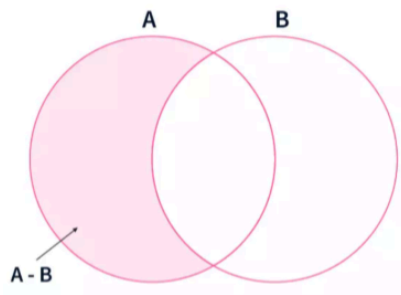
Original changes

Return type = None

In []:

Difference

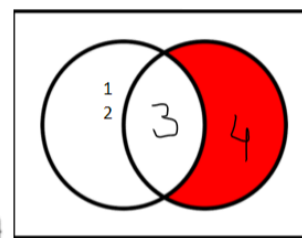
The Difference operation returns elements that are present in the first set but not in the second set.



$$A = \{1, 2, 3\}, B = \{3, 4\}$$

$$A - B = \{1, 2\}$$

Set Difference



$$A = \{1, 2, 3\}, B = \{3, 4\}$$

$$B - A = \{4\}$$

Syntax

Using Operator

Python

`A - B`

Using Method

Python

`A.difference(B)`

In []:

```
In [82]: a = {1, 2, 3, 4, 5}
b = {3, 4, 6}

# Using the difference() method
print(a.difference(b))

# Using the - operator
print(a - b)
```

```
{1, 2, 5}
{1, 2, 5}
```

In [87]: `b - a` # set of elements that are only in B but not in A

Out[87]: {6}

```
In [91]: A = {"Python", "Java", "C++"}
B = {"Java", "C"}

print(A.difference(B))
print(B.difference(A))
```

```
{'Python', 'C++'}
{'C'}
```

```
In [101]: # create four sets
a = {1,2,3,5,8}
b = {2,4,6}
c = {0,1,2,7}
d = {2,3,4}

# elements of set a not in b, c, and d
print(a.difference(b,c,d))
# alternative method
print(a-b-c-d)
```

```
{5, 8}
{8, 5}
```

```
In [99]: # create four sets
a = {1,2,3,5,8}
b = {2,4,6}
c = {0,1,2,7,8}
d = {2,3,4,5}
# elements of set a not in b, c, and d
print(a.difference(b,c,d))
# alternative method
print(a-b-c-d)
```

```
set()
set()
```

difference_update

difference_update() modifies the original set by removing common elements."

```
In [113... # Python code to get the difference between two sets
# using difference_update() between set A and set B

# Driver Code
A = {10, 20, 30, 40, 80}
B = {100, 30, 80, 40, 60}
print(A.difference(B))
print(B.difference(A))

print(A)
```

```
{10, 20}
{100, 60}
{80, 20, 40, 10, 30}
```

```
In [107... # Modifies A and returns None
A.difference_update(B)
```

```
{20, 10}
```

```
In [109... # Prints the modified set
print (A)
```

```
{20, 10}
```

difference()

Returns new set

Original unchanged

Return type = set

difference_update()

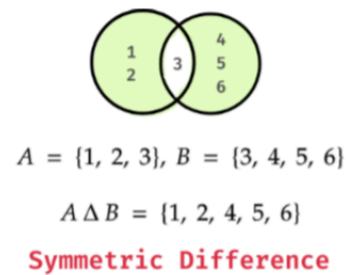
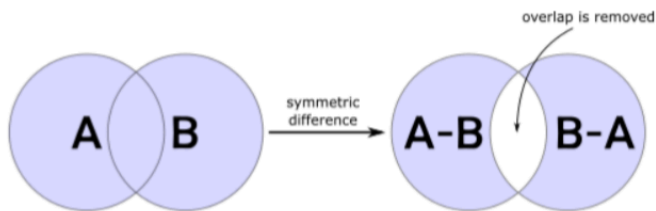
Modifies original set

Original changed

Return type = None

Symmetric Difference

Symmetric Difference returns non-common elements from two sets.



Syntax

Using Operator

Python

$A \wedge B$

Using Method

Python

`A.symmetric_difference(B)`

method

Important Points

✓ Accepts:

- list
- tuple
- string
- set

✗ Does NOT support multiple arguments.

```
In [139... A = {1,2,3}
print(A.symmetric_difference([3,4,5]))#List
print(A.symmetric_difference((3,4)))#tuple
{1, 2, 4, 5}
{1, 2, 4}
```

```
In [141... A.symmetric_difference(B,C)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[141], line 1
----> 1 A.symmetric_difference(B,C)
TypeError: set.symmetric_difference() takes exactly one argument (2 given)
```

```
In [ ]:
```

^ Operator

^ is the symmetric difference operator. It works only with sets.

Important Points

✓ Supports:

- set objects only

✗ Does NOT support:

- list
- tuple
- string

```
In [145... A = {1,2,3}
B = {3,4,5}

print(A ^ B)
```

```
{1, 2, 4, 5}
```

```
In [147... A = {1,2,3}

print(A ^ [3,4,5])
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[147], line 3
      1 A = {1,2,3}
----> 3 print(A ^ [3,4,5])

TypeError: unsupported operand type(s) for ^: 'set' and 'list'
```

```
In [149... A = {1,2}
B = {2,3}
C = {3,4}

print(A ^ B ^ C)
```

```
{1, 4}
```

```
In [ ]:
```

```
In [ ]:
```

```
In [116... a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Using the symmetric_difference() method
print(a.symmetric_difference(b))

# Using the ^ operator
print(a ^ b)
```

```
{1, 2, 5, 6}
{1, 2, 5, 6}
```

```
In [122... set_a = {"apple", "banana", "cherry"}
list_b = ["google", "microsoft", "apple"]

# Finds elements in set_a or list_b, but not both
result = set_a.symmetric_difference(list_b)

print(result)
```

```
{'google', 'microsoft', 'banana', 'cherry'}
```

```
In [124... A={1,2}
B={2,3,5}
C={3,4}

print (A.symmetric_difference(B,C))
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[124], line 5
      2 B={2,3,5}
      3 C={3,4}
----> 5 print (A.symmetric_difference(B,C))

TypeError: set.symmetric_difference() takes exactly one argument (2 given)
```

```
In [126... A={1,2}
B={2,3,5}
C={3,4}

print (A^B^C)#Output:{1,4,5}

{1, 4, 5}

In [128... A = {1,2,3}

print(A.symmetric_difference([3,4,5]))

{1, 2, 4, 5}

In [130... #iterable is given as list
A={1,2,3}
B=[1]
print (A.symmetric_difference(B))#Output:{2,3}

#iterable is given as tuple
A={1,2,3}
B=(1,)
print (A.symmetric_difference(B))#Output:{2,3}

#iterable is given as range object
A={1,2,3}
B=range(2)
print (A.symmetric_difference(B))#Output:{2,3}

{2, 3}
{2, 3}
{0, 2, 3}

In [132... A={1,2,3}
B=[1]
A^B
print (A) #Output: TypeError: unsupported operand type(s) for ^: 'set' and 'list'

-----
TypeError                                Traceback (most recent call last)
Cell In[132], line 3
      1 A={1,2,3}
      2 B=[1]
----> 3 A^B
      4 print (A)

TypeError: unsupported operand type(s) for ^: 'set' and 'list'

In [ ]:
```

symmetric_difference_update()

Updates original set with non-common elements only.

```
In [120... A = {1,2,3}
B = {3,4,5}

A.symmetric_difference_update(B)

print(A)

{1, 2, 4, 5}
```

Feature	symmetric_difference()	symmetric_difference_update()
Returns new set	Yes	
Modifies original set	No	Yes
Original set unchanged	Yes	No
Return value	New set	None
Operation type	Non in-place	In-place

Method	Operator	Return Type	
union()		New set	
update()	=	None	Modifies the original set
intersection()	&	New set	
intersection_update()	&=	None	Modifies the original set
difference()	-	New set	
difference_update()	-=	None	Modifies the original set
symmetric_difference()	^	New set	
symmetric_difference_update()	^=	None	Modifies the original set
isdisjoint()		bool	
issubset()	<=	bool	
issuperset()	>=	bool	
Proper Subset	<	bool	
Proper superset	>	bool	

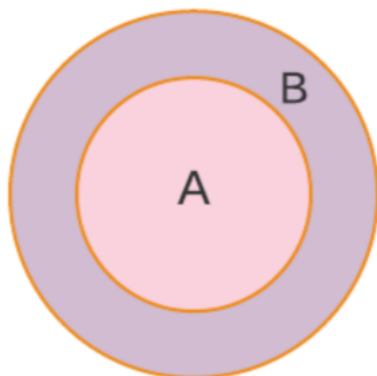
Set Relation Methods

issubset() and issuperset() are called set relationship methods because they compare relationships between sets."

In []:

issubset()

checks whether all elements of one set are present in another set.



In []: *#Check whether set A is a subset of B*

```
In [159... A={1,2,3,4,5}
B={4,5,6,7,8}
print (A.issubset(B)) #Output: False
print (A<=B)#Output: False
```

False
False

```
In [161... A={1,2,3}
B={1,2,3,4,5}
print (A.issubset(B)) #Output: True
print (A<=B)#Output: #true
```

True
True

Proper Subset

```
In [ ]: A = {1,2}
B = {1,2,3}

print(A < B)
```

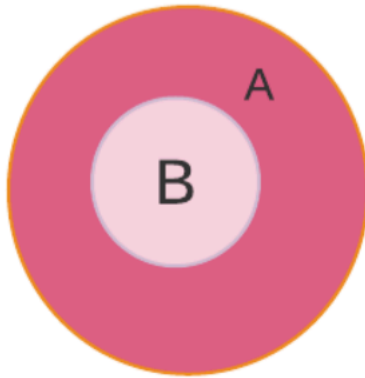
"Subset allows equal sets, whereas proper subset requires the first set to be strictly smaller than the second set."

In []:

superset

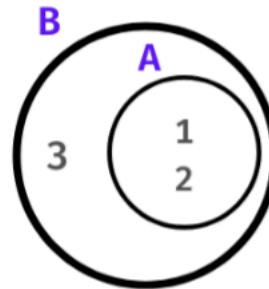
One set contains all elements of another set.

A.issuperset(B) ->True



$$A = \{1, 2\}, B = \{1, 2, 3\}$$

$$B \supset A$$



B superset of A

```
In [15]: # create two sets
a = {1, 2, 3, 4}
b = {1, 2, 3}
# check if a is superset of b
a.issuperset(b)
```

Out[15]: True

```
In [17]: A = {1, 2, 3, 4, 5}
B = {1, 2, 3, 0}

print(A.issuperset(B))
```

False

```
In [4]: x = {1,2,3}
y = {1,2,4}

print(x.issuperset(y))
```

False

```
In [8]: ## Check whether set A is a superset of B
A={1,2,3,4,5}
B={4,5,6,7,8}
print (A.issuperset(B))
print (A>=B)
```

False

False

```
In [6]: A={1,2,3,4,5}
        B={1,2,3}
        print (A.issuperset(B)) #Output: True
        print (A>=B)#Output:True
```

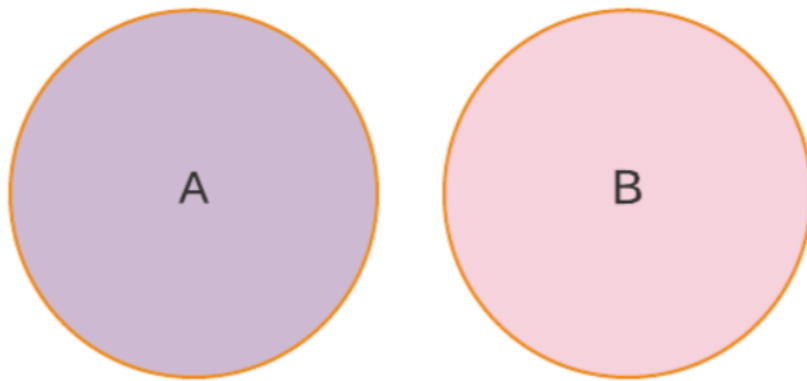
True
True

In []:

isdisjoint()

Returns True if the set has no elements in common with the other. Sets are disjoint if and only if their intersection is the empty set.

A.isdisjoint(B) ->True



```
In [12]: #Set A and Set B containing common elements
        A={1,2,3,4,5}
        B={4,5,6,7,8}
        print (A.isdisjoint(B))

        #Set A and Set B not containing common elements
        A={1,2}
        B={3,4}
        print (A.isdisjoint(B))
```

False
True

In []:

```
In [19]: A = {1,2,3,4,5,6,7,8,9}
        B = {3,4,5,6,7,8}
        C = {10,20,30,40}
```

```
In [21]: B.issubset(A) # Set B is said to be the subset of set A if all elements of B are included A
```

Out[21]: True

```
In [23]: A.issuperset(B)#Set A is called a superset of set B if all elements of B are present in A.
```

Out[23]: True

```
In [ ]: print(C.isdisjoint(A))
        print(B.isdisjoint(A))
```

```
In [25]: print(C.isdisjoint(A))
        print(B.isdisjoint(A))
```

True
False

frozenset

frozenset is an immutable version of a Python set.

Meaning:

Elements cannot be changed after creation

No add()

No remove()

No update()

```
In [31]: fs = frozenset([1,2,3,4])
print(fs)
```

frozenset({1, 2, 3, 4})

```
In [33]: fs = frozenset("python")
print(fs)
```

frozenset({'p', 'o', 'n', 't', 'y', 'h'})

```
In [35]: fs = frozenset("python")
print(fs)
```

frozenset({'p', 'o', 'n', 't', 'y', 'h'})

```
In [37]: s = {
    frozenset([1,2]),
    frozenset([3,4])
}
print(s)
```

{frozenset({3, 4}), frozenset({1, 2})}

```
In [39]: fs = frozenset([1,2,3])
fs.add(4)
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[39], line 3
      1 fs = frozenset([1,2,3])
----> 3 fs.add(4)

AttributeError: 'frozenset' object has no attribute 'add'
```

```
In [41]: A = frozenset([1,2,3])
B = frozenset([3,4,5])
print(A.union(B))
```

frozenset({1, 2, 3, 4, 5})

```
In [43]: A = frozenset([1,2,3])
B = frozenset([2,3,4])
print(A.intersection(B))
```

frozenset({2, 3})

6. Comparison Table: List vs Tuple vs Set vs Dictionary

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ } or set()	{ key: value }
Ordered	Yes	Yes	No	Insertion order (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique, values can repeat
Indexing	Yes	Yes	No	By key only
Use Case	General collection, modification needed	Fixed data, function returns	Unique elements, fast lookup	Key-value mapping, fast retrieval
Performance (Lookup)	O(n)	O(n)	O(1) average	O(1) average

In []:

In []:

1. Why set is unordered?

Because it uses hashing internally.

In []:

2. Why duplicates are not allowed?

Each element must have a unique hash identity.

end

In []: